WEEK 2 · DAY 1 · OVERVIEW

Exponential Smoothing



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Week 1 ended with no model of a series' own past closer to each date's change than the last value carried forward, neither Day 2's drift nor Day 5's ARIMA.

Week 2 opens with the oldest family of all, exponential smoothing: an average that weights the newest row most and lets older rows fade.

RiskMetrics, a risk method published in 1996, runs the same one-line recursion to describe volatility.

Today we fit it to the level of five series, score it walk-forward against Day 2's bar, then point it at the size of a change.

As on every day of this course, each output is a value for a date inside the file from the rows before it, never a forecast of rates or spreads.

OBJECTIVES

Write the exponentially weighted moving average as a loop, match pandas, and read its alpha as weights and a half-life.

Fit simple smoothing, Holt's trend and the damped trend with **`ExponentialSmoothing`**, and read alpha, beta and phi.

Turn a fitted smoother into one-step values that use only earlier rows, and score smoothers walk-forward with MASE beside Day 2's bar and Day 5's AR(1).

Build an EWMA volatility of the daily changes, draw its band, and score the smoothed size of a change.

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 5 EXERCISES

Your Route Through Day 1

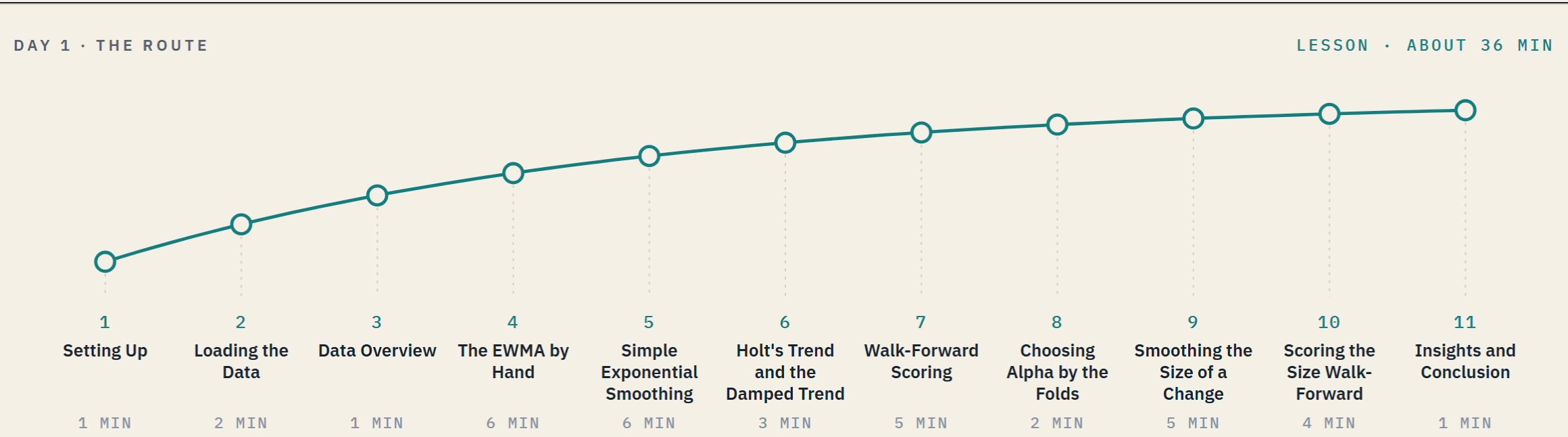


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 36 minutes for the lesson, then about 15 for the five exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import mean_absolute_error
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
```

Loading the Data

READING THE TWO FILES

```
yields = pd.read_csv(DATA + "treasury_par_yields.csv", parse_dates=["date"], index_col="date")
spreads = pd.read_csv(DATA + "course3_synthetic_spreads.csv", parse_dates=["date"], index_col="date")
CUT = "2022-12-30"
development = yields.loc[:CUT]
```

FIVE SERIES IN BP

```
levels = pd.DataFrame({"2 Yr": (development["2 Yr"] * 100).round(2),
                       "10 Yr": (development["10 Yr"] * 100).round(2),
                       "2s10s": ((development["10 Yr"] - development["2 Yr"]) * 100).round(2),
                       "IG": spreads.loc[:CUT, "ig_spread_bp"],
                       "HY": spreads.loc[:CUT, "hy_spread_bp"]})
changes = levels.diff().dropna()
```

Data Overview

THE SHAPE AND THE SCORED DATES

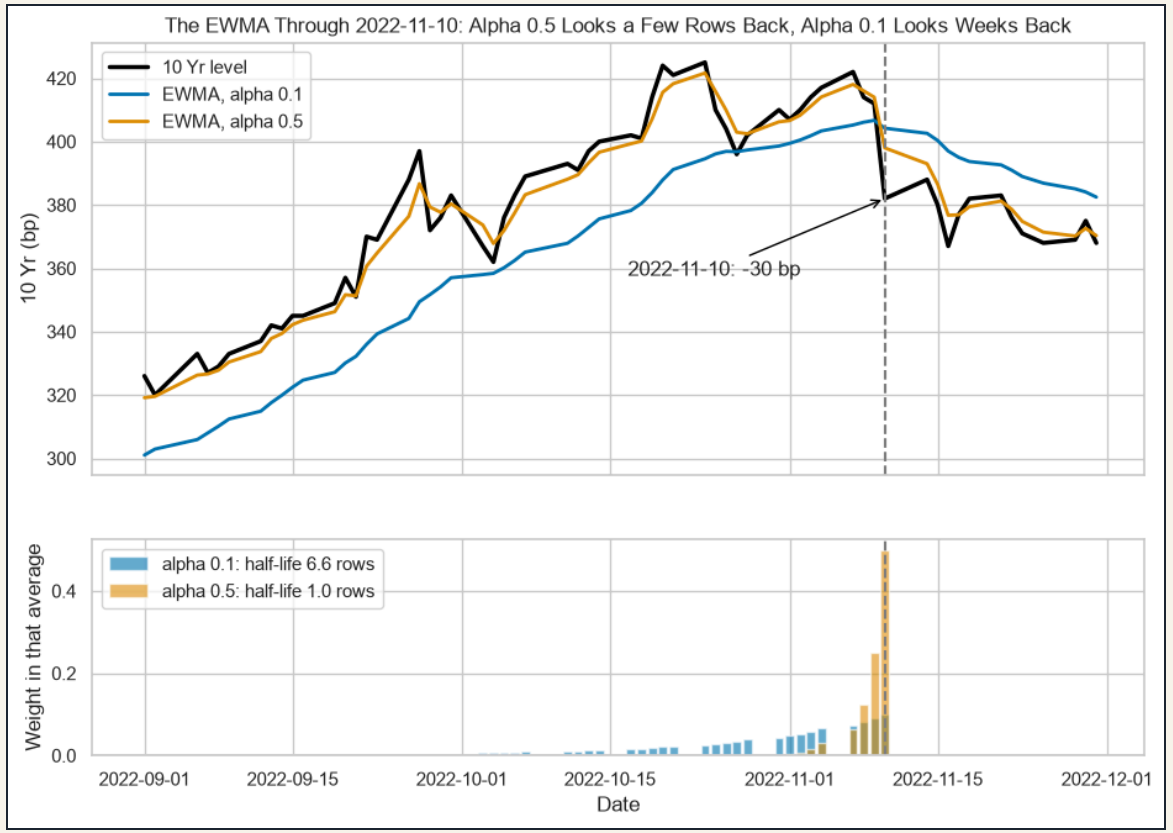
```
Levels: (2001, 5) | changes: (2000, 5) | blank cells: 0
Scored dates: 2017-12-15 to 2022-12-30
```

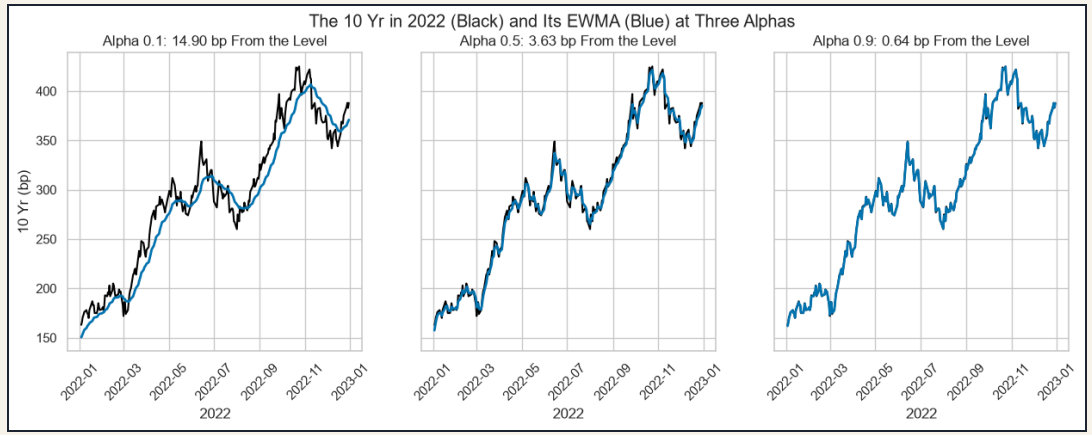
THE BAR, IN BP: EACH FOLD'S LAST-VALUE MAE

	FOLD 1	FOLD 2	FOLD 3	FOLD 4	FOLD 5	AVERAGE
2 Yr	2.0675	3.4881	1.9008	1.4881	6.0675	3.0024
10 Yr	2.6905	3.4444	3.6627	3.3056	6.2579	3.8722
2s10s	1.8214	1.6468	2.7222	2.8016	3.8413	2.5667
IG	0.7349	0.8448	0.7603	1.0115	1.2087	0.9121
HY	3.3321	3.0635	3.2437	5.0127	5.0329	3.9370

Five complete series: the smoothers read the levels, and every score reads the changes.

The EWMA by Hand





THE LOOP

```
ten = levels["10 Yr"].to_numpy()
alpha = 0.2
average = [ten[0]]
for level in ten[1:]:
    average.append(alpha * level + (1 - alpha) * average[-1])
```

The average through a date includes that date; to use it as a value for a date, shift it one row. Which alpha suits a value for a date is for the data to say.

Simple Exponential Smoothing

THE FIT

```
train, test = changes.index[folds[0][0]], changes.index[folds[0][1]]
fold1_levels = levels.loc[:train[-1], "10 Yr"]
simple = ExponentialSmoothing(fold1_levels.to_numpy(), initialization_method="estimated").fit()
```

THE QUESTION

Q. What alpha does least squares give the 10 Yr on those 741 levels? Write down a number between 0 and 1.

THE CUT TEST

```
blanked = levels["10 Yr"].copy()
blanked.loc["2019-12-31":] = np.nan
for shift in [1, 0]:
    whole = levels["10 Yr"].ewm(alpha=alpha_hat, adjust=False).mean().shift(shift).loc[:"2019-12-31"]
    stopped = blanked.ewm(alpha=alpha_hat, adjust=False).mean().shift(shift).loc[:"2019-12-31"]
    print("shift", shift, "| equal:", whole.equals(stopped), "|", len(whole.compare(stopped)), "of", len(whole), "values moved")
```

```
shift 1 | equal: True | 0 of 1250 values moved
shift 0 | equal: False | 1 of 1250 values moved
```

Unshifted, the cut date's own value moves: the average through a date reads that date.

Holt's Trend and the Damped Trend

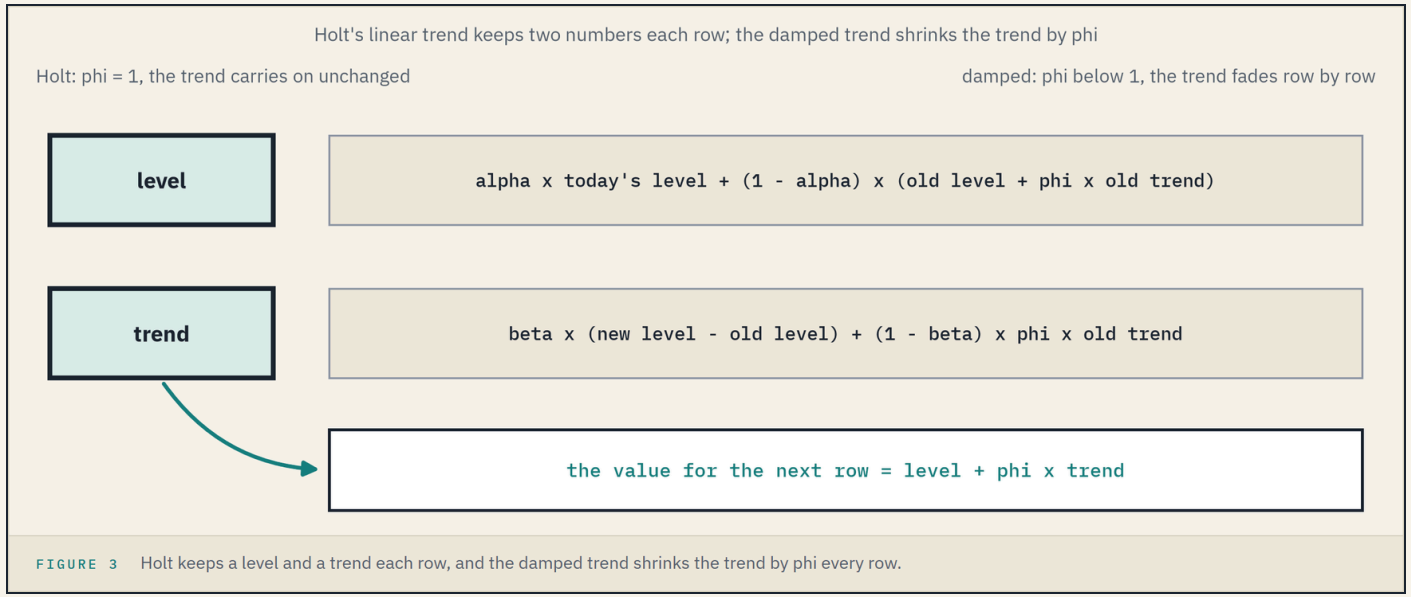


FIGURE 3 Holt keeps a level and a trend each row, and the damped trend shrinks the trend by phi every row.

THE QUESTION

Q. Holt's trend on the same 741 levels of the 10 Yr: what beta does least squares give it? Write down a number between 0 and 1.

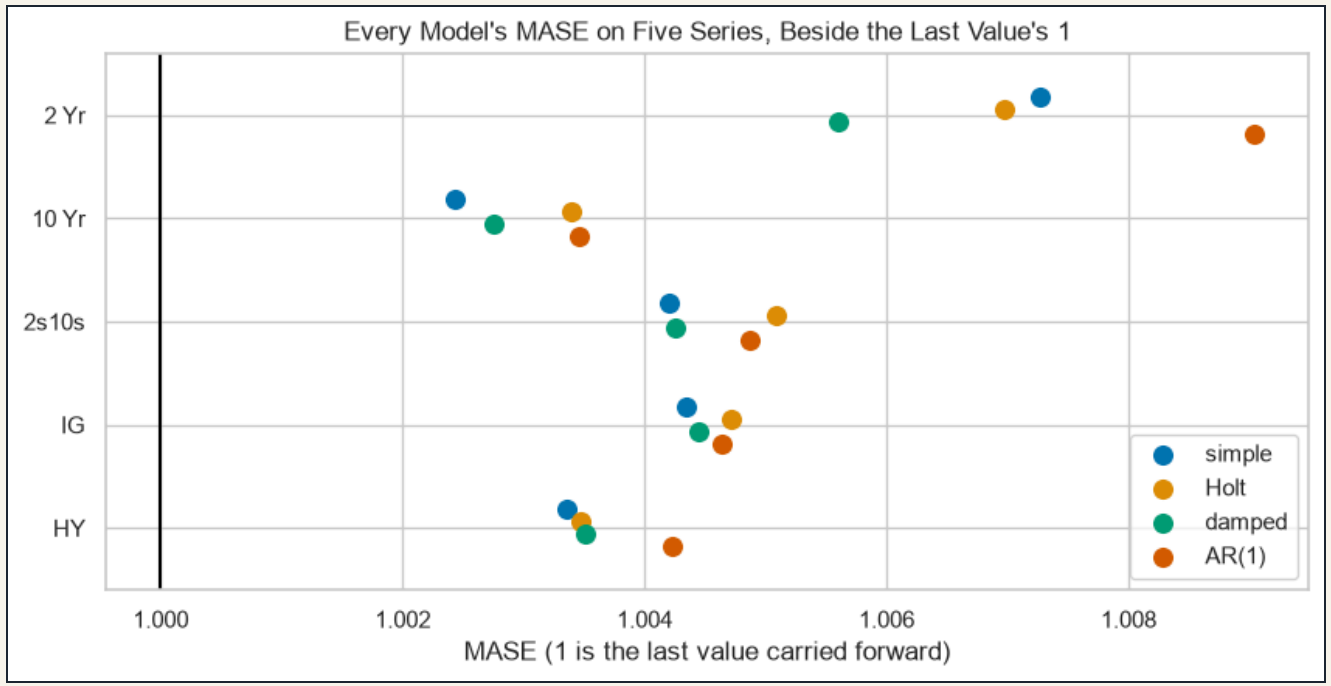
BETA AND PHI

beta is the trend's alpha, the weight a new row's change gets in the trend.

THREE SMOOTHERS ON FOLD 1

```
values = fold1_levels.to_numpy()
fits = {"simple": simple,
        "Holt": ExponentialSmoothing(values, trend="add", initialization_method="estimated").fit(),
        "damped": ExponentialSmoothing(values, trend="add", damped_trend=True, initialization_method="estimated").fit()}
```

Walk-Forward Scoring



MASE OF EACH MODEL

MODEL	SIMPLE	HOLT	DAMPED	AR(1)
series				
2 Yr	1.0073	1.0070	1.0056	1.0090
10 Yr	1.0024	1.0034	1.0027	1.0035
2s10s	1.0042	1.0051	1.0043	1.0049
IG	1.0043	1.0047	1.0044	1.0046
HY	1.0034	1.0035	1.0035	1.0042

By Day 2's rule no smoother beats the last value on any of the five series: every gap is far inside the folds' disagreement, and most go the wrong way.

Hold each alpha fixed and score all five.

Choosing Alpha by the Folds

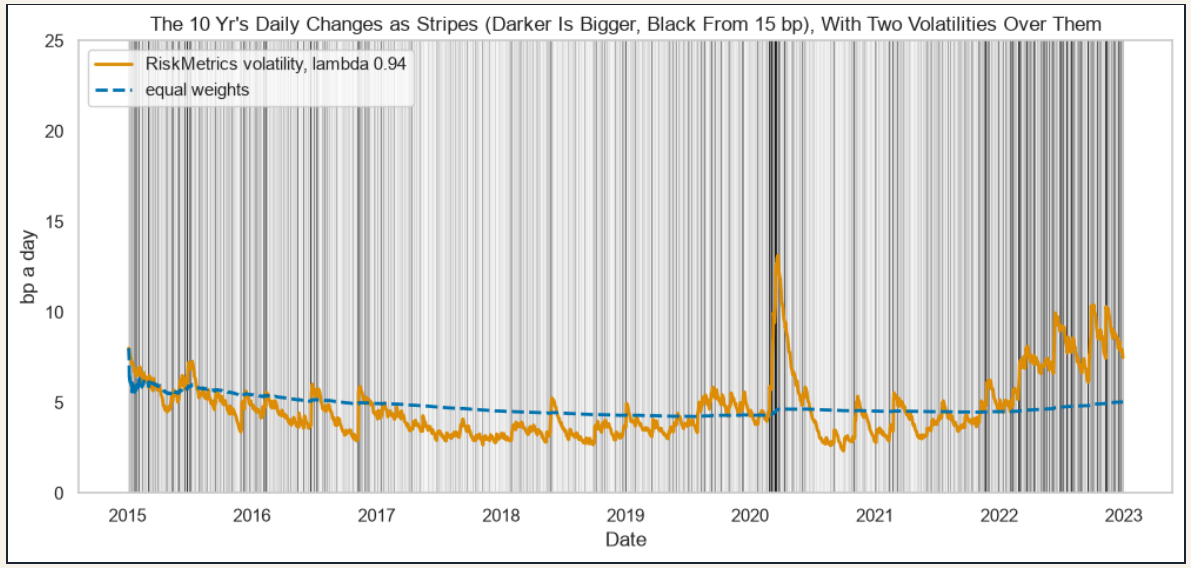
A FIXED ALPHA AS A RULE

```
scored = changes.index[folds[0][1][0]:]
grid = [0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9, 1.0]
fixed_mae, fixed_size = {}, {}
for a in grid:
    fitted = (levels.ewm(alpha=a, adjust=False).mean().shift(1) - levels.shift(1)).loc[scored]
    fixed_mae[a] = (fitted - changes.loc[scored]).abs().mean()
    fixed_size[a] = fitted.abs().mean()
fixed_gap = pd.DataFrame(fixed_mae).T - bar["average"]
fixed_gap
```

THE RULE

A fixed alpha needs no fit: its one-step value is the EWMA through the row before, so it scores every fold at once.

Smoothing the Size of a Change



THE SIZE

The square root of that average is an **EWMA volatility**, in bp a day.

THE QUESTION

Q. For each year from 2015 to 2022, sketch how many dates fall outside the band, once with equal weights and once with lambda 0.94.

THE EWMA VOLATILITY

```
riskmetrics = np.sqrt(squares.ewm(alpha=1 - LAMBDA, adjust=False).mean())
equal = np.sqrt(squares.expanding().mean())
print("Half-life at lambda 0.94:", round(np.log(0.5) / np.log(LAMBDA), 1), "rows")
print("Through 2022-12-30: RiskMetrics", round(riskmetrics.iloc[-1], 2), "bp a day | equal weights", round(equal.iloc[-1], 2), "bp a day")
```

```
Half-life at lambda 0.94: 11.2 rows
Through 2022-12-30: RiskMetrics 7.49 bp a day | equal weights 5.03 bp a day
```

Score the smoothed size against two rules.

Scoring the Size Walk-Forward

THE QUESTION

Q. The EWMA of the size at lambda 0.94, through the row before each date: what is its MAE over the five folds, in bp? Write down a number.

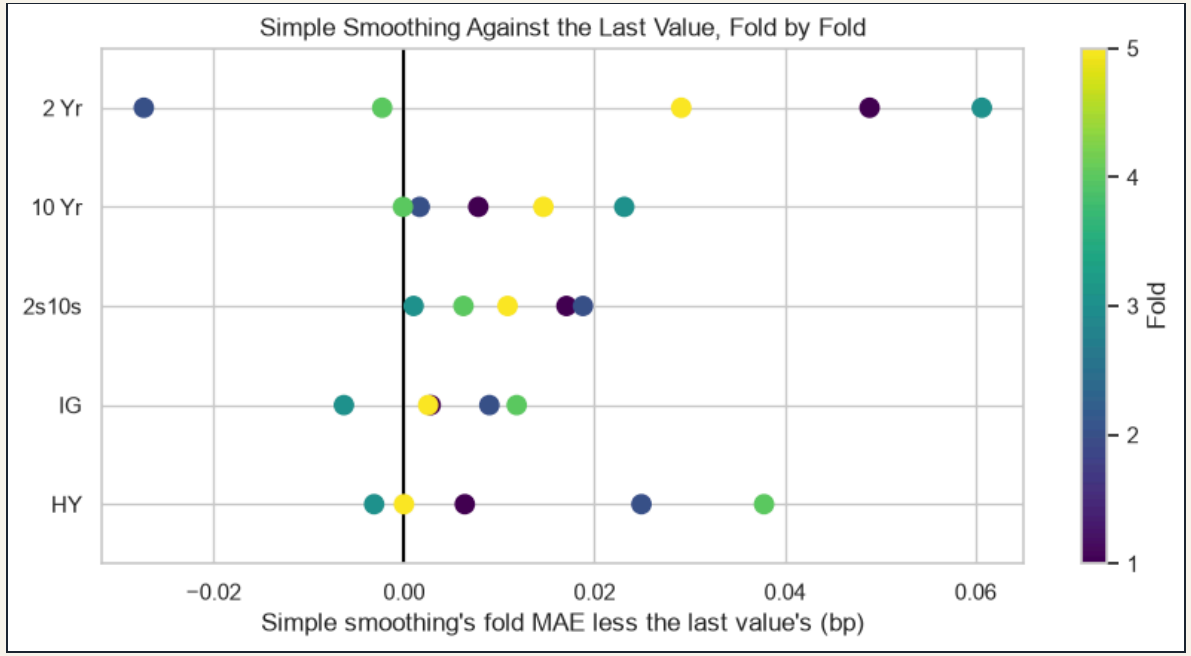
TWO RULES FOR THE SIZE

```
size_rows = []
for fold, (fit_rows, test_rows) in enumerate(folds, start=1):
    train, test = changes.index[fit_rows], changes.index[test_rows]
    size_rows.append({"fold": fold, "training mean": mean_absolute_error(size[test], [size[train].mean()] * len(test)),
                      "last value": mean_absolute_error(size[test], size.shift(1)[test])})
size_scores = pd.DataFrame(size_rows).set_index("fold")
size_scores.mean()
```

THE EWMA OF THE SIZE

```
smoothed_size = size.ewm(alpha=1 - LAMBDA, adjust=False).mean().shift(1)
size_scores["lambda 0.94"] = [mean_absolute_error(size.iloc[test], smoothed_size.iloc[test]) for train, test in folds]
size_scores
```

Insights and Conclusion



SIMPLE SMOOTHING LESS THE LAST VALUE, IN BP

FOLD	1	2	3	4	5
2 Yr	0.0489	-0.0273	0.0607	-0.0023	0.0291
10 Yr	0.0078	0.0016	0.0231	-0.0001	0.0146
2s10s	0.0171	0.0188	0.0010	0.0062	0.0109
IG	0.0028	0.0090	-0.0063	0.0118	0.0025
HY	0.0064	0.0249	-0.0031	0.0378	-0.0000

Every smoother's MASE runs from 1.0024 to 1.0073, inside the folds' disagreement.

You Can Now

- Write an EWMA as a loop and with `.ewm(alpha=..., adjust=False).mean()`, and read alpha as a half-life.
- Fit simple smoothing, Holt's trend and the damped trend with `ExponentialSmoothing`, and read alpha, beta and phi.
- Turn a smoother into one-step values with `.shift(1)`, or with a refit whose weights are held, and pass the cut test.
- Score smoothers walk-forward with MASE beside the last value and an AR(1).
- Build an EWMA volatility of daily changes, draw its band, and score the smoothed size of a change.

NEXT
WEEK 2 · DAY 2

Volatility Clustering and GARCH

Day 7 fits GARCH, a model of the size of a change.